👌 XAI: LIME & SHAP

FAMAF - UNC

First Semester 2026

Disclaimer ---

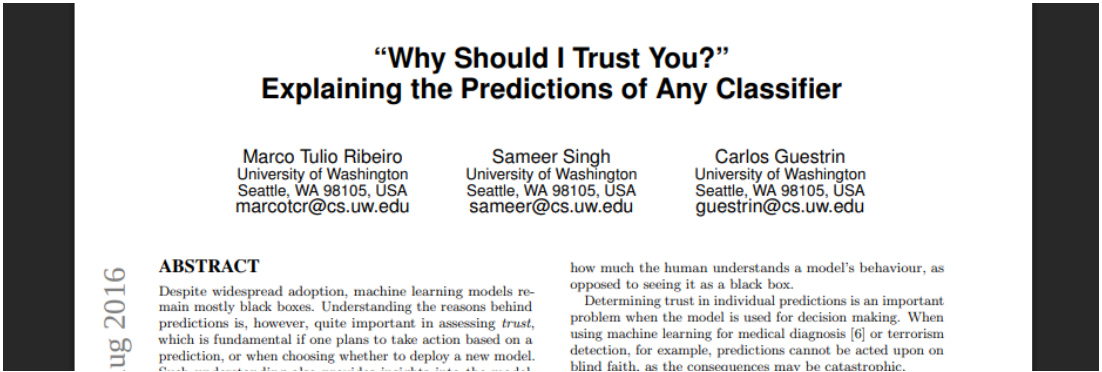
This course is based on

Explainable Artificial Intelligence

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

https://interpretable-ml-class.github.io/



"Why Should I Trust You?" Explaining the Predictions of Any Classifier

Marco Tulio Ribeiro
University of Washington
Seattle, WA 98105, USA
marcotcr@cs.uw.edu

Sameer Singh
University of Washington
Seattle, WA 98105, USA
sameer@cs.uw.edu

Carlos Guestrin
University of Washington
Seattle, WA 98105, USA
guestrin@cs.uw.edu

ABSTRACT

Despite widespread adoption, machine learning models remain mostly black boxes. Understanding the reasons behind predictions is, however, quite important in assessing *trust*, which is fundamental if one plans to take action based on a prediction, or when choosing whether to deploy a new model.

how much the human understands a model's behaviour, as opposed to seeing it as a black box.

Determining trust in individual predictions is an important problem when the model is used for decision making. When using machine learning for medical diagnosis [6] or terrorism detection, for example, predictions cannot be acted upon on blind faith, as the consequences may be catastrophic.

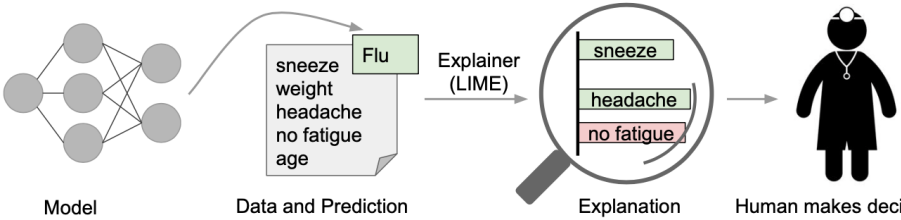
(Ribeiro, Singh, and Guestrin 2016)

Define Trust - Two Notions ---

ML models are often used as black boxes — but humans need to understand them before acting on their outputs.

- **Trusting a prediction:** Does this specific output make sense? *E.g., should a doctor act on this diagnosis?*
- **Trusting a model:** Will this model behave reasonably when deployed? *E.g., does it generalize beyond the validation set?*
  Both depend on how much the user understands the model's behavior.

Define Explanation for a Prediction ---



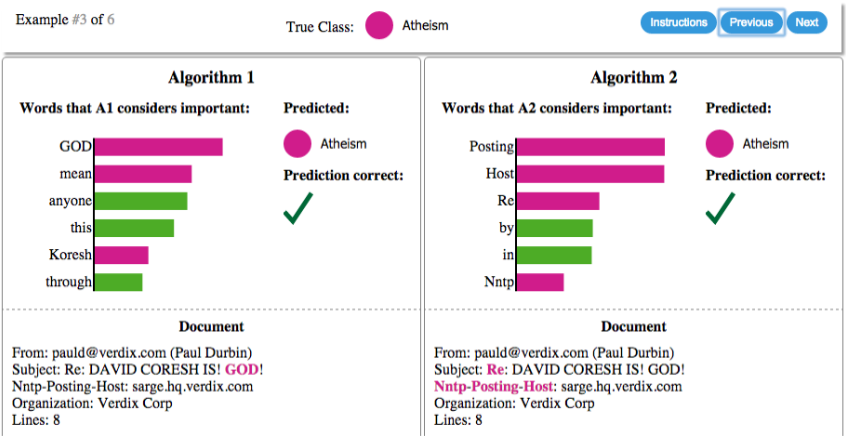
An explanation identifies **which parts of the input** drove the model's prediction — and in which direction.

- 🟢 *sneeze*, *headache* → evidence **for** Flu
- 🔴 *no fatigue* → evidence **against** Flu

Formally:

A textual or visual artifact that describes the relationship between an instance's components and the model's output.

Explanations as a Means to Select Model 1/2 ---



Both models predict "Atheism" correctly — but for very different reasons.

Explanations as a Means to Select Model 2/2 ---

Both models predict "Atheism" correctly — but for very different reasons.

	Algorithm 1	Algorithm 2
Key words	*GOD, mean, anyone*	*Posting, Host, Nntp*
Reason	Semantic content ✅	Email header metadata ❌
Will it generalize?	Likely yes	Likely no

- Accuracy alone cannot distinguish these models.
- Explanations expose *why* a prediction was made — and whether to trust it.

Why Do We Need Trust and Interpretability? ---

A model can have **high accuracy** and still be wrong for the right reasons.

- **Data leakage:** the model learns features that are accidentally correlated with the label (e.g., patient ID predicts diagnosis). High accuracy, zero generalization.
- **Dataset shift / Concept drift:** training and real-world data differ — either the input distribution changes/shift, or the relationship between inputs and labels changes/drift over time. The model looks good on validation but fails in deployment.
- **Spurious features:** the model exploits features that *work* statistically but are undesirable (e.g., clickbait signals in a recommender system).
  In all three cases, validation accuracy is misleading. Interpretability lets us catch these failures before they cause harm.

Desired Characteristics for Explainers ---

- **Interpretable:** explanations must be understandable to humans, not just technically correct (e.g., no thousands of non-zero weights)
- **Locally faithful:** must reflect how the model *actually behaves* near the instance being explained — global fidelity is often impossible
- **Model-agnostic:** must work as a black box, applicable to any classifier, including models that don't yet exist
- **Global perspective:** beyond single predictions, explanations should help users understand the model as a whole

Local Interpretable Model-agnostic Explanations (LIME)

A framework designed to satisfy all four criteria simultaneously.

LIME Step 1: Define LIME Framework ---

Goal: Find the simplest explanation that still accurately describes the model's behavior *near* the instance x.

$$\xi(x) = \underset{g \in G}{\textbf{argmin}}\ \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

- $\xi(x)$ — explanation for instance x
- $g \in G$ — interpretable model (e.g., linear model)
- f — black-box model to be explained
- $\mathcal{L}(f, g, \pi_x)$ — unfaithfulness of g approximating f locally
- π_x — proximity measure that defines the local neighborhood
- $\Omega(g)$ — complexity of g (e.g., number of non-zero weights)

This is a **fidelity-interpretability trade-off**: minimize error locally, while keeping the explanation simple enough for humans to understand.

LIME Step 2: Interpretable Data Representation ---

- The black-box model operates on features that are often incomprehensible to humans (e.g., embeddings, raw pixels).
- LIME maps these to an **interpretable representation** that humans can reason about:
  - **Text:** binary vector indicating the presence or absence of a word
  - **Image:** binary vector indicating the presence or absence of a contiguous patch of similar pixels (superpixel)

$$x \in \mathbb{R}^d \xrightarrow{h_x} x' \in \{0,1\}^{d'}$$

Original Representation $\longrightarrow$ Interpretable Representation

where $x \in \mathbb{R}^d$ is the original instance with d features, $x' \in \{0,1\}^{d'}$ is a binary vector of length d' where each entry is 1 if the corresponding interpretable component is **present** and 0 if it is **absent**, and h_x is the function that maps x' back to the original representation.

LIME Step 3: Sampling for Local Exploration 1/2 ---

Since f is a black box, LIME **interrogates** it: perturb, query, and fit g locally.

1. Start from x', the interpretable representation of the instance to explain
2. Generate perturbed samples z' by randomly turning off components of x'
3. Map each z' back to the original space: $z = h_x(z')$
4. Query the black-box model to get a label: $f(z)$
5. Weight each sample by its proximity to x: $\pi_x(z)$

$$x \to x' \xrightarrow{\text{perturb}} z' \xrightarrow{h_x} z \xrightarrow{f} f(z)$$

The dataset $\mathcal{Z} = \{(z', f(z), \pi_x(z))\}$ is then used to fit the local model g.

LIME Step 3: Sampling for Local Exploration 2/2 ---

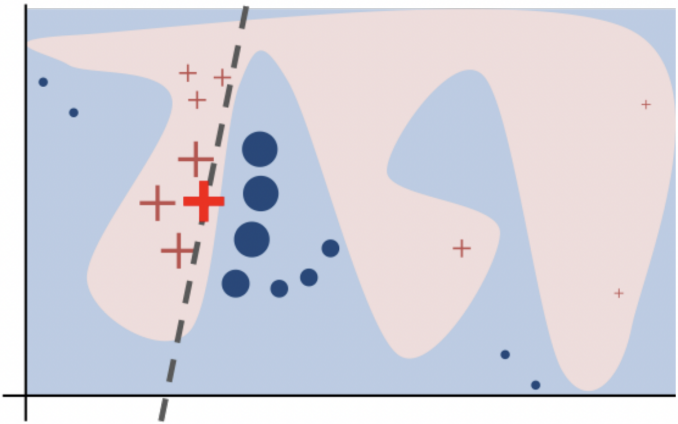
Since f is a black box, LIME cannot minimize $\mathcal{L}(f, g, \pi_x)$ analytically. Instead, it **interrogates** f: it generates perturbed samples near x', queries the model, and uses the responses to fit g locally.

1. **Start from the instance:** take the review *"The movie was great and funny"* $\rightarrow x' = [1, 1, 1, 1, 1, 1]$
2. **Randomly turn off words:** generate a masked version $\rightarrow z' = [0, 1, 0, 1, 0, 1] \rightarrow$ *"movie great funny"*
3. **Map back to original space:** reconstruct the input the model understands $\rightarrow z = h_x(z')$ (e.g., recompute embeddings for *"movie great funny"*)
4. **Ask the black box:** what does f predict for this masked input? $\rightarrow f(z) = 0.85$ (positive sentiment)
5. **Weight by proximity:** samples closer to x matter more $\rightarrow \pi_x(z)$ is high if few words were removed, low otherwise

$$x \rightarrow x' \xrightarrow{\text{perturb}} z' \xrightarrow{h_x} z \xrightarrow{f} f(z)$$

Repeat N times $\rightarrow$ fit a sparse linear model g on $\mathcal{Z} = \{(z', f(z), \pi_x(z))\}$.

Sampling Procedure Intuition ---



A complex decision boundary is approximated locally by a simple linear model (dashed line). Samples closer to the instance of interest (red cross) are weighted more.

Example: Sparse Linear Explanation ---

$$\xi(x) = \underset{g \in G}{\operatorname{argmin}} \ \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

$$g(z') = w_g \cdot z'$$

$$\pi_x(z) = \exp\left(-\frac{D(x, z)^2}{\sigma^2}\right)$$

$$\mathcal{L}(f, g, \pi_x) = \sum_{z, z' \in \mathcal{Z}} \pi_x(z)\big(f(z) - g(z')\big)^2$$

$$\Omega(g) = \infty \mathbb{1}[\|w_g\|_0 > K]$$

Algorithm 1 Sparse Linear Explanations using LIME

Require: Classifier f, Number of samples N
Require: Instance x, and its interpretable version x'
Require: Similarity kernel π_x, Length of explanation K
  $\mathcal{Z} \leftarrow \{\}$
  for $i \in \{1, 2, 3, ..., N\}$ **do**
    $z'_i \leftarrow sample\_around(x')$
    $\mathcal{Z} \leftarrow \mathcal{Z} \cup \langle z'_i, f(z_i), \pi_x(z_i) \rangle$
  end for
  $w \leftarrow$ K-Lasso$(\mathcal{Z}, K)$ ▷ with z'_i as features, $f(z)$ as target
  return w

Example: Sparse Linear Explanation ---

$$\xi(x) = \underset{g \in G}{\operatorname{argmin}} \ \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

$$g(z') = w_g \cdot z'$$

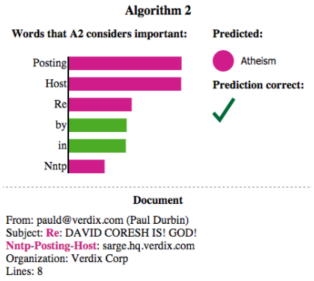
$$\pi_x(z) = \exp\left(-\frac{D(x,z)^2}{\sigma^2}\right)$$

$$\mathcal{L}(f, g, \pi_x) = \sum_{z, z' \in \mathcal{Z}} \pi_x(z)\left(f(z) - g(z')\right)^2$$

$$\Omega(g) = \infty \mathbb{1}[\|w_g\|_0 > K]$$

```python
def lime_explain(f, x, x_prime, pi_x, K, N):
    """
    f       : black-box model (callable)
    x       : original instance
    x_prime : interpretable representation of x (binary vector)
    pi_x    : proximity function centered at x
    K       : max number of features in explanation
    N       : number of samples
    """
    Z = []
    for _ in range(N):
        # Step 1: perturb x' by randomly turning off components
        z_prime = random_mask(x_prime)

        # Step 2: map back to original representation
        z = h_x(z_prime, x)

        # Step 3: query the black-box model
        label = f(z)

        # Step 4: compute proximity weight
        weight = pi_x(z)

        Z.append((z_prime, label, weight))

    # Step 5: fit sparse linear model with at most K features
    g = k_lasso(Z, K)

    return g
```

Some Results ---



(a) Original Image (b) Explaining *Electric guitar* (c) Explaining *Acoustic guitar* (d) Explaining *Labrador*

Figure 4: Explaining an image classification prediction made by Google's Inception neural network. The top 3 classes predicted are "Electric Guitar" ($p = 0.32$), "Acoustic guitar" ($p = 0.24$) and "Labrador" ($p = 0.21$)

Global LIME: Intuition ---

If we could explain **every instance** in the dataset, we would have a complete picture of how the model behaves globally.

But this is clearly infeasible — datasets can have thousands of instances, and users have limited time and patience.

Key question:

Which B instances should we show the user to maximize their understanding of the model?

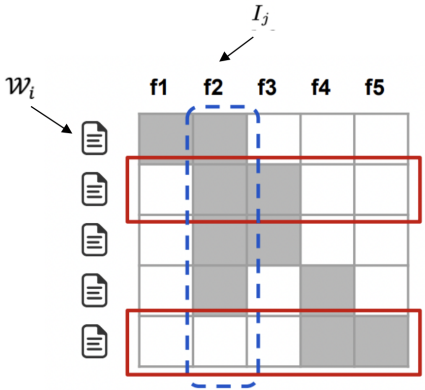
Gain Global Understanding of the Model ---

A single explanation gives local insight — but how do we understand the model **globally**? The pick step selects B instances to show the user, such that together they are maximally informative about the model's overall behavior.

- **Budget B:** users have limited time — we can only show B explanations
- **Explanation-aware selection:** instances are chosen based on their LIME explanations, not raw data alone — looking at raw predictions is not enough
- **Diverse and representative:** selected instances should cover as many different important features as possible, avoiding redundancy
  The goal: with just B explanations, give the user a global picture of how the model works.

Submodular Pick (SP) Algorithm 1/2 ---

After running LIME on every B instance, we organize the results into a matrix $\mathcal{W}_{n \times d'}$ that summarizes how important each interpretable feature is across all instances.



- Each **row** i represents an instance x_i
- Each **column** j represents an interpretable feature (e.g., a word)
- Each **cell** $\mathcal{W}_{ij} = |w_{g_j}|$ is the importance LIME assigned to feature j for instance i — 0 if unused

From $\mathcal{W}$, the **global importance** of each feature is:

$$I_j = \sqrt{\sum_{i=1}^{n} |\mathcal{W}_{ij}|}$$

A high I_j means feature j is relevant across many instances.

Submodular Pick (SP) Algorithm 2/2 ---

Algorithm 2 Submodular pick (SP) algorithm

Require: Instances X, Budget B
  for all $x_i \in X$ **do**
    $\mathcal{W}_i \leftarrow$ **explain**(x_i, x_i') ▷ Using Algorithm 1
  end for
  for $j \in \{1 \dots d'\}$ **do**
    $I_j \leftarrow \sqrt{\sum_{i=1}^{n} |\mathcal{W}_{ij}|}$ ▷ Compute feature importances
  end for
  $V \leftarrow \{\}$
  while $|V| < B$ **do** ▷ Greedy optimization of Eq (4)
    $V \leftarrow V \cup \text{argmax}_i\, c(V \cup \{i\}, \mathcal{W}, I)$
  end while $Pick(\mathcal{W}, I)$
  return V

We want to select a set V of at most B instances that **maximizes coverage** of globally important features:

$$c(V, \mathcal{W}, I) = \sum_{j=1}^{d'} \mathbb{1}_{[\exists i \in V : \mathcal{W}_{ij} > 0]} I_j$$

$$\text{Pick}(\mathcal{W}, I) = \underset{V, |V| \leq B}{\text{argmax}}\ c(V, \mathcal{W}, I)$$

c is a **submodular function**: adding a new instance to V yields diminishing returns as V grows.

Greedy solution: at each step, add the instance that maximally increases coverage:

Simulated User Experiment - Experimental Setup ---

Goal: evaluate explanation quality across different models and methods.

Datasets: two sentiment analysis datasets (books and DVDs, 2000 instances each) — task is to classify product reviews as positive or negative.

Models trained: Decision Tree, Logistic Regression, Nearest Neighbors, SVM, Random Forest

Explanation methods compared:

- **Random:** selects K features at random — minimal baseline
- **Greedy:** removes features one by one until the prediction changes
- **Parzen:** approximates f globally with Parzen windows, explains via gradient
- **LIME:** proposed method — local sparse linear approximation

Instance selection strategies:

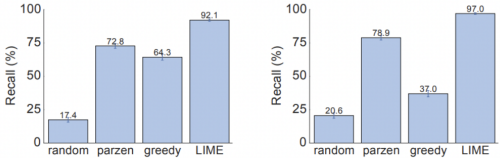
- **Random Pick (RP):** selects instances at random
- **Submodular Pick (SP):** selects instances to maximize feature coverage

All methods produce explanations of size $K = 10$ features.

Simulated User Experiment - Are explanations faithful to the method? ---

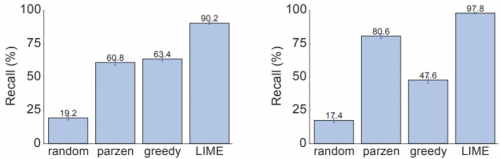
Key idea: use interpretable models (sparse LR, decision trees) where we *know* the true important features — the **gold set**. Then measure how many gold features each method recovers (**recall**).

LIME achieves the highest recall of truly important features on both Books and DVDs datasets.



(a) Sparse LR (b) Decision Tree

Figure 6: Recall on truly important features for two interpretable classifiers on the books dataset.



(a) Sparse LR (b) Decision Tree

Figure 7: Recall on truly important features for two interpretable classifiers on the DVDs dataset.

Simulated User Experiment - Should I trust this prediction? ---

- Randomly mark 25% of features as untrustworthy (simulating problematic features such as data leakage or spurious correlations). A prediction is labeled **untrustworthy** if it changes when those features are removed (the model was relying on them).
- The prediction is then untrustworthy if the problematic features appear in it with high weight.
- We measure **F1 of trustworthiness** to evaluate whether each explanation method correctly identifies which predictions to trust and which to reject.

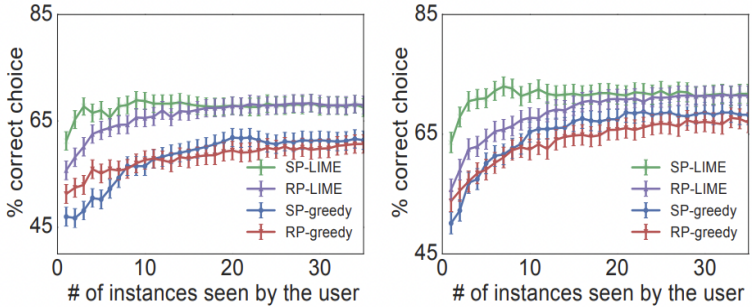
Table 1: Average F1 of *trustworthiness* for different explainers on a collection of classifiers and datasets.

	Books				DVDs			
	LR	NN	RF	SVM	LR	NN	RF	SVM
Random	14.6	14.8	14.7	14.7	14.2	14.3	14.5	14.4
Parzen	84.0	87.6	94.3	92.3	87.0	81.7	94.2	87.3
Greedy	53.7	47.4	45.0	53.3	52.4	58.1	46.6	55.1
LIME	**96.6**	**94.5**	**96.2**	**96.7**	**96.6**	**91.8**	**96.1**	**95.6**

LIME maintains both high precision and high recall — it neither over-trusts nor over-distrusts predictions.

Simulated User Experiment - Can I trust this model? ---

- 10 noisy features added: correlated with the label in train/validation, but not in test
- Two random forests with similar validation accuracy ($< 0.1\%$ apart) but very different test accuracy ($> 5\%$ apart)
- **Task:** can a simulated user pick the better model by inspecting B explanations?
- **SP-LIME** dominates, especially for small B



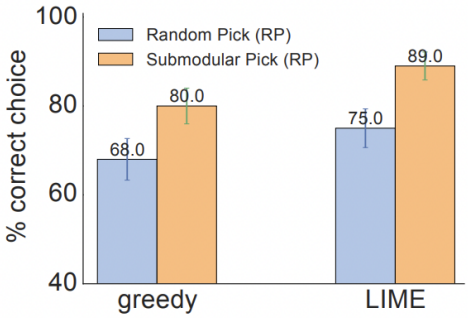
Evaluating with Human Subjects - Experimental Setup ---

Previous experiments were simulated — now we test with **real users** on Amazon Mechanical Turk.

- **Dataset:** 20 Newsgroups (Christianity vs. Atheism) — known to contain spurious features (headers, author names) that do not generalize
- To measure real-world generalization, a **new religion dataset** is created:
  - 819 web pages per class scraped from Atheism and Christianity websites
- Model: SVM with RBF kernel, hyperparameters tuned via cross-validation

Evaluating with Human Subjects - Can users select the best classifier? ---

- Train two SVMs:
  - one on problematic dataset
  - one on "cleaned" dataset
- Recruit humans to select which algorithm will perform best
- The problematic model has *higher* validation accuracy (94.0% vs 88.6%), so accuracy alone would lead to the wrong choice.

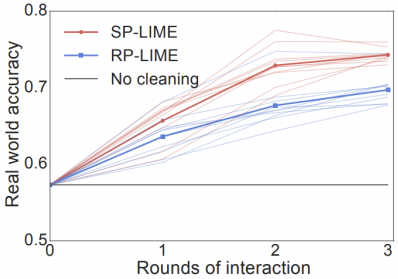


SP-LIME (89%) outperforms RP-LIME (75%), and both outperform greedy variants.

Evaluating with Human Subjects - Can non-experts improve a classifier? ---

Each round: user sees $B = 10$ instances with $K = 10$ words per explanation and marks words to remove. Model is retrained without those words.

- Round 0: 10 subjects $\rightarrow$ 10 classifiers
- Round 1: 5 new users per classifier $\rightarrow$ 50 classifiers
- Round 2: 5 more users $\rightarrow$ 250 classifiers
- Real-world accuracy measured at each round on the religion dataset



Result

Non-experts using LIME explanations can meaningfully improve a classifier without ever seeing the test data.

Evaluating with Human Subjects - Do explanations lead to insights? ---

Setup: a deliberately bad classifier is trained to predict "wolf" when there is snow in the background, and "husky" otherwise — ignoring the actual animal.

1. Find pictures such that classifier predicts "wolf" if there is snow, and "husky" otherwise
2. Ask subjects (grad students):
   - Do they trust model to work well in real world?
   - Why?
   - How do they think the model distinguishes?
3. Showed explanations and asked again

(a) Husky classified as wolf (b) Explanation

Figure 11: Raw data and explanation of a bad model's prediction in the "Husky vs Wolf" task.

	Before	After
Trusted the bad model	10 out of 27	3 out of 27
Snow as a potential feature	12 out of 27	25 out of 27

Related Work ---

Inherently Interpretable Models

- Trees, lists, sets, (generalized) linear models, and additive models
- *Interpretable Decision Sets* (Lakkaraju et al.) — joint framework for description and prediction

Post-hoc, Model-agnostic

- LIME: one of the **first significant papers** for post-hoc, model-agnostic analysis
- *Extracting Faithful and Descriptive Representations* — approximating globally through gradient is a challenge $\Rightarrow$ Local fidelity

Show, Attend and Tell (Xu et al.)

Interpretable description of images via visual attention — LIME aims for a more model-agnostic approach.

Limitations and Future Work ---

- **Feature space complexity:** expected to work on feature spaces that are not too complex
  - Can similar ideas be applied to sequential decision making (e.g., RL)?
- **User experiment:** results could depend on how the study notified the definition of "trustworthiness" to participants
- **Automation:** a principled way for the algorithm to automatically improve based on its explanation (without humans having to modify the data or algorithm manually)

A Unified Approach to Interpreting Model Predictions

Scott M. Lundberg
Paul G. Allen School of Computer Science
University of Washington
Seattle, WA 98105
slund1@cs.washington.edu

Su-In Lee
Paul G. Allen School of Computer Science
Department of Genome Sciences
University of Washington
Seattle, WA 98105
suinlee@cs.washington.edu

Abstract

Understanding why a model makes a certain prediction can be as crucial as the prediction's accuracy in many applications. However, the highest accuracy for large modern datasets is often achieved by complex models that even experts struggle to interpret, such as ensemble or deep learning models, creating a tension between *accuracy* and *interpretability*. In response, various methods have recently been proposed to help users interpret the predictions of complex models, but it is often unclear how these methods are related and when one method is preferable over

(Lundberg and Lee 2017)

Additive Feature Attribution Methods (AFAMs) ---

This paper unifies **6 previous methods** for local interpretability under a single framework. An AFAM explains a prediction $f(x)$ by assigning an importance ϕ_i to each feature i:

$$g(z') = \phi_0 + \sum_{i=1}^{M} \phi_i z'_i, \quad z' \in \{0,1\}^M$$

where g is a linear explanation model over the interpretable representation z'.

Example: LIME for image classification

- Features of x are its **superpixels**
- Removing a feature = replacing its superpixel with grey pixels
- y = the all-grey image (input with no features)
- LIME fits $g(z') \approx f(x)$ and $g(y') \approx f(y)$ locally
- The weights ϕ_i indicate how important each superpixel was for $f(x)$

The key insight:

many existing methods already fit this form — often without realizing they share the same underlying structure.

DeepLIFT as an Additive Feature Attribution Method ---

Instead of perturbing features randomly, DeepLIFT compares the current prediction against a **reference input** r (e.g., a blank or average image).

- Features of x are its **pixels**
- Removing a feature = replacing pixel x_i with its value in r
- $\phi_0 = f(r)$ — base prediction on the reference input
- Each $\phi_i = C_{\Delta x_i \Delta o}$ measures how much pixel i contributed *relative to its reference value*

DeepLIFT satisfies the **summation-to-delta** property exactly:

$$\sum_{i=1}^{n} C_{\Delta x_i \Delta o} = f(x) - f(r)$$

Human mode: The attributions sum exactly to the difference between the current prediction and the reference prediction — every unit of "change" (pixel) in the output is fully accounted for by the input features.

LIME vs DeepLIFT ---

- Unlike LIME, DeepLIFT is not model-agnostic — it exploits the internal structure of neural networks to back-propagate attribution values in a single pass, without any sampling.

DeepLIFT is an AFAM:

Setting

$$\phi_0 = f(r)$$

and

$$\phi_i = C_{\Delta x_i \Delta o}$$

makes it fit exactly the linear form

$$g(z') = \phi_0 + \sum_i \phi_i z'_i$$

Desiderata for Additive Feature Attribution Methods ---

Three desirable properties for an AFAM:

Local Accuracy: the attributions must fully account for the prediction — summing all ϕ_i plus the base value ϕ_0 must recover $f(x)$ exactly. No part of the prediction should be left unexplained.

Consistency: if feature i always contributes more in model f than in model f' — regardless of what other features are present — then $\phi_i(f) \geq \phi_i(f')$. Intuitively: if a feature becomes more important, its attribution should not decrease. LIME can violate this depending on the sampling.

Missingness: if a feature was not present in the original input ($x'_i = 0$), its attribution must be zero — you cannot credit or blame something that never existed.

Key result:

These three properties together determine a *unique* solution — **SHAP values**.

The Shapley Value Problem 1/3 ---

Three workers collaborate to earn a reward: **carpenter (C), painter (P), salesperson (S)**.

Coalition	Reward
{C}	$10
{P}	$15
{S}	$5
{C, P}	$40
{C, S}	$30
{P, S}	$35
{C, P, S}	$100

Problem: how much did each worker actually contribute to the $100 reward?

The Shapley Value Problem 2/3 ---

Coalition	Reward
{C}	$10
{P}	$15
{S}	$5
{C, P}	$40
{C, S}	$30
{P, S}	$35
{C, P, S}	$100

Problem: how much did each worker actually contribute to the $100 reward?

- The contribution of each worker depends on who else is cooperating — the salesperson adds $60 when both C and P are present, but only $20 when only C is present.

Shapley answer

Average each player's marginal contribution over all possible coalitions.

The Shapley Value Problem 3/3 ---

The value of player i is:

$$\phi_i = \sum_{S \subseteq [d] \setminus \{i\}} \frac{|S|!(d - |S| - 1)!}{d!} [g(S \cup \{i\}) - g(S)]$$

For the **salesperson (S)** whit $d = 3$:

Coalition S	$g(S \cup \{S\}) - g(S)$	Weight $\frac{\|S\|!(d-\|S\|-1)!}{d!}$	Contribution
$\emptyset$	$5 - 0 = 5$	$\frac{0! \cdot 2!}{3!} = \frac{2}{6}$	1.67
{C}	$30 - 10 = 20$	$\frac{1! \cdot 1!}{3!} = \frac{1}{6}$	3.33
{P}	$35 - 15 = 20$	$\frac{1! \cdot 1!}{3!} = \frac{1}{6}$	3.33
{C, P}	$100 - 40 = 60$	$\frac{2! \cdot 0!}{3!} = \frac{2}{6}$	20

From Games to Local Interpretability ---

Idea: treat features as players and the model prediction $f(x)$ as the reward. Shapley values would then tell us how much each feature contributed to the prediction.

Problem: to compute marginal contributions, we need to evaluate the model on every possible subset of features S — but the model was trained on all features at once.

What would the model have predicted if it only had access to features S?

For example, if $x = [\text{age} = 30, \text{income} = 50k, \text{debt} = 10k]$ and we want to evaluate using only $\{\text{age}, \text{income}\}$ — what value do we give to debt?

$$g(S) = f(x_S) \quad \text{is not well-defined}$$

The model cannot handle arbitrary patterns of missing inputs.

This is the key challenge SHAP must solve.

Previously Proposed: Separate Models ---

- In **"Shapley regression values"**, for every subset of features $S \subseteq [d]$, we can train a model f_S that tries to predict the labels
- The resulting Shapley values are a good metric for how important each feature is for good prediction of the labels
- However, they do **not** provide interpretability for the specific model f we were working with
  - What f might do without any information about the features in $[d] \setminus S$ might be very different than optimal prediction

Human mode

Shapley regression values measure which features are important for predicting the target well, not which features your specific model relies on. They are useful for global feature selection, but not for explaining why f made a particular prediction.

Feature Ablation ---

We want to capture what our **particular** model f would do if it had no access to features $[d] \setminus S$.

This notion is captured by the **expectation of the model output given features S**:

$$E[f(x) \mid x_S] \tag{3}$$

We do not remove the missing features — instead we **neutralize** them by averaging f's predictions over different possible values, sampled from their real distribution. The model still receives all its inputs, but the missing features no longer carry information.

We can approximate this by sampling over the conditional distribution:

$$\frac{1}{N} \sum_{i=1}^{N} f(x^{(i)}), \quad x^{(i)} \sim x \mid x_S \tag{4}$$

Feature Ablation Example 1/2 ---

Suppose $x = [\text{age} = 30, \text{income} = 50k, \text{debt} = 10k]$ and $S = \{\text{age, income}\}$. We want to estimate what f predicts when it has no access to `debt`.

Instead of removing `debt`, we sample N values from its conditional distribution — i.e., examples in the dataset with age=30 and income=50k — and evaluate f for each:

- $f([30, 50k, 10k])$
- $f([30, 50k, 5k])$
- $f([30, 50k, 20k])$
- ...

Then we average the results. This gives us what f predicts on average when it only knows age and income, without privileging any particular value of `debt`.

Feature Ablation Example 2/2 ---

With $N = 3$ samples and $S = \{\text{age, income}\}$:

$$E[f(x) \mid x_S] \approx \frac{1}{3}\left[f([30, 50k, 10k]) + f([30, 50k, 5k]) + f([30, 50k, 20k])\right]$$

Suppose the model predicts:

- $f([30, 50k, 10k]) = 0.8$
- $f([30, 50k, 5k]) = 0.7$
- $f([30, 50k, 20k]) = 0.6$

Then:

$$E[f(x) \mid x_S] \approx \frac{0.8 + 0.7 + 0.6}{3} = \frac{2.1}{3} = 0.7$$

This is our estimate of what f predicts when it only has access to age=30 and income=50k, neutralizing the effect of `debt`.

Model Agnostic: Feature Independence ---

- Computing SHAP values requires calculating 2^d differences $g(S \cup \{i\}) - g(S)$
  - With $d = 20$ features, that amounts to over one million model evaluations — infeasible in practice.
- One way to improve: assume **features are independent**, $\forall S$:

$$E[f(x) \mid x_S] = E_{x_{\bar{S}}|x_S}[f(x)] \approx E_{x_{\bar{S}}}[f(x)] \tag{5}$$

- We can then estimate SHAP values via sampling approximations (**the Shapley sampling values method**) which require fewer than 2^d difference calculations
  - Following the previous example, we simply sample values of `debt` from the entire dataset, regardless of age or income.
- But still requires lots of computations

Kernel SHAP: LIME with the Right Parameters 1/2 ---

- LIME's parameters **(π_x, L, Ω)** were chosen heuristically — they work well in practice but **do not guarantee** local accuracy or consistency.
- **Key insight:** there exists a unique choice of these parameters such that the solution to LIME's regression is exactly the SHAP values:

$$\Omega(g) = 0 \qquad \pi_x(z') = \frac{(M-1)}{\binom{M}{|z'|}|z'|(M-|z'|)} \qquad L(f, g, \pi_x) = \sum_{z' \in \mathcal{Z}} \left[f_h(z') - g(z')\right]^2 \pi_x(z')$$

- $\Omega(g) = 0$ — no regularization: SHAP values do not penalize complexity
- $\pi_x(z')$ — upweights very small and very large coalitions, downweights intermediate ones
- L — standard weighted squared loss, same as LIME

Kernel SHAP: LIME with the Right Parameters 1/2 ---

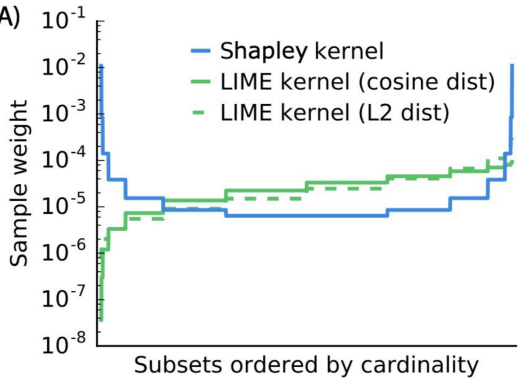
$$\Omega(g) = 0 \qquad \pi_x(z') = \frac{(M-1)}{\binom{M}{|z'|}|z'|(M-|z'|)} \qquad L(f, g, \pi_x) = \sum_{z' \in \mathcal{Z}} \left[f_h(z') - g(z')\right]^2 \pi_x(z')$$

Why this kernel?

Coalitions of 1 or $M-1$ features are the most informative — we know exactly which feature is being added or removed. Intermediate coalitions are more ambiguous because many features change at once.

Kernel SHAP = LIME + Shapley kernel
Satisfies local accuracy and consistency by construction.

Kernel SHAP: Sample Weight Comparison ---



The Shapley kernel assigns more weight to very small and very large subsets compared to LIME kernels (cosine or L2 distance).

Kernel SHAP: Linear Approximation ---

Instead of sampling missing features, we can assume f is linear and simply replace each missing feature with its expected value $E[x_i]$:

$$x_i^* = \begin{cases} x_i & \text{if } i \in S \\ E[x_i] & \text{otherwise} \end{cases}$$

Example: $x = [\text{age} = 30, \text{income} = 50k, \text{debt} = 10k]$, $S = \{\text{age}, \text{income}\}$

Instead of sampling `debt`, we substitute $E[\text{debt}] = 15k$ directly:

$$E[f(x) \mid x_S] \approx f([30, 50k, 15k]) = 0.75$$

Since g is linear and L is a squared loss, all ϕ_i can be solved **jointly** via weighted linear regression — more efficient than estimating each separately.

Kernel SHAP = LIME's regression framework + Shapley kernel + mean imputation

Model-Specific Approximations: Linear SHAP 1/2 ---

For **linear (affine) models**, SHAP values can be computed analytically — no sampling or regression needed.

$$\textbf{If} \quad f(x) = \sum_{j=1}^{M} w_j x_j + b \qquad \textbf{Then} \qquad \phi_0(f, x) = b \qquad \phi_i(f, x) = w_i(x_i - E[x_i])$$

Intuition: feature i's contribution is how far its value deviates from the average, scaled by how much the model cares about it (w_i).

- If $x_i = E[x_i] \rightarrow$ feature i contributes nothing: $\phi_i = 0$
- If $x_i > E[x_i]$ and $w_i > 0 \rightarrow$ positive contribution
- If $x_i < E[x_i]$ and $w_i > 0 \rightarrow$ negative contribution

Model-Specific Approximations: Linear SHAP 2/2 (Example) ---

$f(x) = 2 \cdot \text{age} + 3 \cdot \text{income} + 5,$

with:

- $E[\text{age}] = 40,$
- $E[\text{income}] = 50k,$ and
- $x = [\text{age} = 30, \text{income} = 60k]$:

$$\phi_0 = 5, \quad \phi_{\text{age}} = 2 \cdot (30 - 40) = -20, \quad \phi_{\text{income}} = 3 \cdot (60k - 50k) = 30k$$

Local accuracy check:

$\phi_0 + \phi_{\text{age}} + \phi_{\text{income}} = 5 - 20 + 30 = 15 = f(x)$

Model-Specific Approximations: Deep SHAP ---

DeepLIFT approximates SHAP values under two assumptions: **feature independence** and **model linearity** (via linearization of non-linear components).

Specifically, DeepLIFT:

1. **Linearizes** non-linear components of the network — but the linearization rules were chosen heuristically, without theoretical justification
2. **Replaces missing features** with a reference value, which we interpret as $E[x]$

This means DeepLIFT satisfies **local accuracy** and **missingness**, but **not consistency** — its heuristic linearizations can violate it.

Deep SHAP fixes this by choosing linearizations that are consistent with Shapley values, turning DeepLIFT into a theoretically grounded approximation of SHAP.

Deep SHAP = DeepLIFT + Shapley-consistent linearizations

User Experiments ---

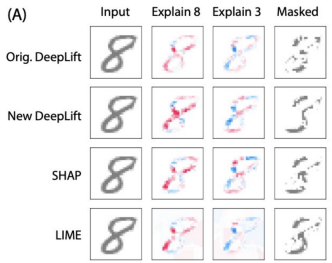
Participants were told a story about a cooperative game and asked to assign credit among players. **Human assignments aligned more closely with SHAP values than with LIME or DeepLIFT.**

Caveat

This experiment uses a cooperative game setting — very different from real neural network attribution. It seems selected to favor SHAP, since SHAP values are derived directly from cooperative game theory. This is weak evidence that SHAP aligns with human intuition for NN predictions.

Class Difference Experiments ---

- Using an image of an “8” and an MNIST classifier, the authors identified which pixels (according to SHAP, LIME, and DeepLIFT) are most important for the model’s log-odds (i.e. logit difference) of 8 versus 3
- **Removing the pixels identified by SHAP** produced larger changes in log-odds from 8 to 3 than the other methods



Extensions: Shapley Values for the Whole Model 1/2 ---

So far SHAP values explain a **single prediction** $f(x)$. Can we explain the **model's behavior globally**?

Naive approach: average SHAP values over all inputs:

$$g(S) = E[E[f(x) \mid x_S]] = E[f(x)]$$

This collapses to a constant by the law of total expectation — it tells us nothing about feature importance.

Extensions: Shapley Values for the Whole Model 1/2 ---

Better approach: measure how much of the model's variance features in S can explain:

$$g(S) = \mathbf{Var}(E[f(x) \mid x_S])$$

- If S contains **important features** $\rightarrow$ knowing x_S makes predictions vary a lot $\rightarrow$ high variance
- If S contains **irrelevant features** $\rightarrow$ knowing x_S barely changes predictions $\rightarrow$ variance ≈ 0

We can then apply Shapley values to $g(S)$ to fairly attribute **how much each feature contributes to the model's overall variance**.

Implementations

SAGE (Covert et al., 2020) and **Shapley Effects** (Owen, 2014).

Extensions: Neuron Shapley ---

- We can **treat neurons as features** (Ghorbani and Zou, 2020)
- This paper uses **zero ablation**
- We can also use **multi-armed bandit sampling algorithms** to estimate Shapley values

Neuron Shapley extends SHAP concepts to understand the contribution of individual neurons in neural networks.

Thank You!

References --- I

Lundberg, Scott M, and Su-In Lee. 2017. “A Unified Approach to Interpreting Model Predictions.” In *Advances in Neural Information Processing Systems*. Vol. 30.

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. 2016. “"Why Should I Trust You?": Explaining the Predictions of Any Classifier.” In *Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining*, 1135–44.